

[**CPK**] Solutions
Independent Security & Systems Risk Analysis

Security Advisory

CPK-2026-002

Hardcoded Tron Private Key Exposure in Production JavaScript Bundle

Platform	GemPad (gempad.app)
Severity	CRITICAL (CVSS 9.1)
CWE	CWE-798 (Use of Hard-coded Credentials), CWE-321 (Use of Hard-coded Cryptographic Key)
Affected Component	_next/static/chunks/pages/_app-4446b1e51d05e6a6.js
Discovered	2026-02-07
Reporter	Christopher Patrick Kuntz

Prepared for the GemPad Security Team
February 2026

Executive Summary

A Tron (TRX) private key and TronGrid API key are hardcoded in GemPad's production JavaScript bundle, publicly accessible to any visitor. The private key corresponds to wallet TQjrBrzXXWRrCXRWZxf6x5xqVrdgJwcXg5, which has deployed 4 smart contracts to Tron mainnet and was directly funded by GemPad's primary Tron deployer wallet (TAGj6vcSQvgiCLeAm5mhcQouQmydxSiKNB), which itself has deployed 49+ contracts and remains active.

This is not a test key. This is a production infrastructure key with on-chain deployment history, exposed in a public JavaScript bundle on a DeFi launchpad that handles user funds.

Finding 1: Hardcoded Tron Private Key

SEVERITY CRITICAL	CVSS SCORE 9.1	DIFFICULTY LOW
------------------------------------	---------------------------------	---------------------------------

Description

The file `_next/static/chunks/pages/_app-4446b1e51d05e6a6.js` contains a hardcoded TronWeb initialization block:

```
new TronWeb({
  fullHost: "https://api.trongrid.io",
  headers: {"TRON-PRO-API-KEY": "6fb3ff76...[REDACTED]"},
  privateKey: "b9205a3a...[REDACTED]...d5add12123"
}).isAddress(e)
```

While the TronWeb instance is only used for `.isAddress()` (an address validation utility that does not require a private key), the private key is nonetheless transmitted to every user's browser and can be extracted trivially.

On-Chain Forensics

The private key resolves to wallet address **TQjrBrzXXWRrCXRWZxf6x5xqVrdgJwcXg5**.

Wallet Activity (50 transactions total)

Category	Count	Details
Contract Deployments	27 attempts (4 successful)	Production smart contracts on Tron mainnet
TRX Transfers	19	Funding and operational transfers
Asset Transfers	4	TRC10/TRC20 token movements

Deployed Contracts (4 live on Tron mainnet)

Contract Address	Deployed	Status
TVFFMNij75aVxTWfFZSYkcqJ6f1Y9pakxd	2024-09-03	Live, unverified source
TGLbvmesfmCsg7K5SdyL515ZVwqjY4tmwa	2024-09-03	Live, unverified source
TRJNTmHGgosH6WfWgdcHYqs95pA2kfEft5	2024-09-02	Live, unverified source
TFSTdbFAxGiQCMxZbpwCXmkh82X8j2UeuR	2024-09-02	Live, unverified source

All four contracts are unverified on TronScan (no published ABI or source code), which prevents independent audit of their admin functions.

Funding Chain: GemPad Infrastructure Link

TAGj6vcSQvgiCLeAm5mhcQouQmydxSiKNB (GemPad Primary Deployer)

49+ contract deployments, active through Dec 2025

| 1,000 TRX funding (2024-08-26)



TQjrBrzXXWRrCXRWZxf6x5xqVrdgJwcXg5 (Exposed Sub-Deployer)

← **PRIVATE KEY IN JS BUNDLE**

27 deployment attempts, 4 successful

Additional 700 TRX + 450 TRX operational transfers

| 394.8 TRX drained to TU4MtifHgsXTYV8iXdYP3m1thuZNoTW9LW (2024-10-14)



Current balance: 0.000002 TRX + 100,000 TREN (test token, 2 holders)

The parent wallet TAGj6vcSQvgiCLeAm5mhcQouQmydxSiKNB has deployed 49+ contracts and was active as recently as December 2025, confirming this is part of GemPad's live Tron infrastructure. This is not a disposable test setup.

Exploit Scenario

1. Attacker visits **https://gempad.app** and opens browser DevTools
2. Searches JavaScript sources for **privateKey**. Finds the 64-character hex key immediately.
3. Imports the key into any Tron-compatible wallet (TronLink, TronWeb CLI)
4. Attacker now controls wallet **TQjrBrzXXWRrCXRWZxf6x5xqVrdgJwcXg5** with full signing authority

With control of the deployer wallet, the attacker can:

- **Call admin functions** on any of the 4 deployed contracts where the deployer retains owner or admin privileges (standard Solidity/TVM pattern: msg.sender in constructor = owner)
- **Deploy new contracts** that appear to originate from GemPad's infrastructure, enabling social engineering or phishing campaigns impersonating GemPad
- **Sign arbitrary transactions** as GemPad's deployer, which could be used to manipulate trust in GemPad-associated addresses
- **Extract any future funds** sent to this wallet (e.g., if it receives fees, refunds, or is referenced in any contract's admin/fee recipient)

Difficulty: Low. No authentication bypass or exploit chain required. The private key is in a publicly accessible JavaScript file. Extraction requires only a web browser.

Finding 2: Hardcoded TronGrid API Key

SEVERITY LOW	IMPACT SERVICE DEGRADATION	DIFFICULTY LOW
-----------------	-------------------------------	-------------------

Description

The same TronWeb initialization block contains a TronGrid API key:

```
TRON-PRO-API-KEY: 6fb3ff76...[REDACTED]
```

TronGrid API keys are free and primarily used for rate limiting, but hardcoding them in client JavaScript allows:

- **Rate limit abuse:** An attacker can exhaust GemPad's API quota, causing service degradation for legitimate users
- **Usage tracking:** TronGrid usage analytics are tied to this key, meaning attacker requests are attributed to GemPad

Recommendations

Immediate (within 24 hours)

5. **Rotate the exposed private key.** Assume it is compromised. Any contracts where TQjrBrzXXWRrCXRWZxf6x5xqVrdgJwcXg5 is the owner must have ownership transferred to a new, unexposed address via `transferOwnership()` or equivalent admin function.
6. **Remove the private key from the JavaScript bundle.** TronWeb's `.isAddress()` is a pure utility function that does not require a private key. Initialize TronWeb without one:

Before (vulnerable):

```
new TronWeb({
  fullHost: "https://api.trongrid.io",
  headers: {"TRON-PRO-API-KEY": "..."},
  privateKey: "b9205a3a...[REDACTED]...d5add12123"
}).isAddress(e)
```

After (safe):

```
TronWeb.isAddress(e)
```

or:

```
new TronWeb({fullHost: "https://api.trongrid.io"}).isAddress(e)
```

TronWeb.isAddress() is a static method that performs local validation only. No network connection or private key is needed.

7. **Rotate the TronGrid API key.** Generate a new key at `trongrid.io` and load it from a server-side environment variable, not the client bundle.

Short-Term (within 7 days)

8. **Audit all 4 deployed contracts** for admin functions accessible by the exposed deployer address. Specifically check for:
 - owner(), admin(), or similar access control state
 - transferOwnership(), renounceOwnership()
 - withdraw(), emergencyWithdraw(), drain()
 - pause(), unpause()
 - upgradeTo(), setImplementation() (proxy patterns)
 - setFeeRecipient(), setAdmin(), setOperator()
9. **Verify the source code** of all 4 contracts on TronScan to enable public auditability.
10. **Audit the parent deployer wallet** (TAGj6vcSQvgiCLeAm5mhcQouQmydxSiKNB) to ensure no other private keys are exposed in client-side code or source control.

Long-Term

11. **Implement secret scanning in CI/CD** to prevent hardcoded keys from reaching production. Tools: gitleaks, trufflehog, or GitHub secret scanning.
12. **Adopt a hardware wallet or multi-sig** for contract deployment operations. A single private key should never be the sole authority for deploying or administering production smart contracts on a DeFi platform.
13. **Implement key management best practices:** server-side signing, environment-variable injection, and separate keys for development vs. production environments.

Scope and Methodology

This finding was discovered during a passive (non-intrusive) external security assessment of gempad.app. The assessment included:

- **JavaScript bundle analysis:** Static analysis of publicly served JavaScript files
- **Secret detection:** Automated scanning using gitleaks and TruffleHog
- **On-chain forensics:** Public blockchain data analysis via TronGrid API and TronScan
- **Key derivation:** Private key → public address derivation using secp256k1 + Keccak-256 to confirm the wallet identity

No private systems were accessed. All data referenced in this advisory is publicly available: the JavaScript bundle is served to every visitor, and all blockchain transactions are on the public Tron ledger.

Disclosure Timeline

Date	Event
2026-02-07 16:12 UTC	Vulnerability discovered via automated scan (CPK Scanner, passive mode)
2026-02-07 16:35 UTC	Private key derived, wallet address confirmed via TronGrid API
2026-02-07 16:58 UTC	On-chain forensics completed: 4 deployed contracts, funding chain traced to GemPad primary deployer
2026-02-07 17:XX UTC	Advisory drafted and sent to security@gempad.app and @GemPadTechSupport (Telegram)
2026-05-08	Disclosure deadline: 90 days from initial report

This advisory follows the disclose.io Core Terms for responsible vulnerability disclosure. We adhere to a 90-day disclosure window, after which the advisory may be published regardless of remediation status, consistent with industry standard practice (Google Project Zero, CERT/CC, Trail of Bits).

References

- [CWE-798: Use of Hard-coded Credentials \(cwe.mitre.org/data/definitions/798.html\)](https://cwe.mitre.org/data/definitions/798.html)
- [CWE-321: Use of Hard-coded Cryptographic Key \(cwe.mitre.org/data/definitions/321.html\)](https://cwe.mitre.org/data/definitions/321.html)
- OWASP Testing Guide: Testing for Credentials in Source Code
- TronWeb Documentation: `isAddress()` (developers.tron.network)
- [disclose.io Core Terms \(github.com/disclose/dioterms\)](https://github.com/disclose/dioterms)
- GemPad security.txt (gempad.app/.well-known/security.txt)

This advisory is reported in good faith under responsible disclosure principles. No funds were moved, no contracts were interacted with, no authentication was bypassed, and no automated exploitation was performed. All analysis was conducted using publicly available data.

[**CPK**] Solutions

Automated Web3 Security Assessment

christopher@cpk.solutions

cpk.solutions